

BARTHOLOMEW

The Deterministic In-Memory Execution Gate for Autonomous AI Agents

Product: btp-guard (Python & TypeScript)

Live Sandbox: <https://bartholomew.info>

GitHub: <https://github.com/ivegotahunnitonit/bartholomew>

Stage: Pre-Seed / Seed (YC 2026 Batch Application)

The Problem: Autonomous Agents Cannot Be Trusted

Autonomous tool-calling agents (CrewAI, LangGraph, AutoGen, Claude Computer Use) are ready to run business workflows. However, security teams block production rollout because LLMs hallucinate destructive commands:

- **Catastrophic Shell Commands:** Agents running `rm -rf /`, disk formatters, or modifying system configs.
- **Production Data Destruction:** Unchecked DDL commands like `DROP TABLE users`; or unbounded updates.
- **Credential Exfiltration:** LLMs accidentally embedding AWS keys, OpenAI keys, or JWT tokens into public APIs.
- **Runaway Token Bills:** Recursive tool loops burning thousands of dollars in minutes.

Current Dilemma: Cloud guardrails (AWS Bedrock, NeMo) add 1.2s to 2.5s latency per tool call and fail against prompt injections. Full microVMs (Docker) are too heavy for local agent execution.

The Solution: In-Memory AST Invariant Enforcement

Bartholomew provides **pre-flight, in-process deterministic validation** for agent tool calls. Instead of asking another probabilistic LLM for permission, Bartholomew parses proposed code blocks in memory:

Sub-50 Microsecond Speed

Executes compiler-level AST checks in $<38 \mu\text{s}$ directly in Python/Node memory. 30,000x faster than cloud LLM judges.

Zero Cloud Overhead

\$0.00 cost per check. 100% local, requiring zero outbound HTTP requests, background daemons, or tokens.

Cryptographic Attestations

Generates RFC 8785 Ed25519 tamper-proof digital signatures for every executed action for SOC 2 & EU AI Act audits.

1-Line Drop-In SDK: Defense-in-Depth

Developers secure any agent framework with a single decorator:

```
from btp_guard import secure_tool

@secure_tool(strict_mode=True)
def run_terminal_command(cmd: str):
    # Catches 'rm -rf' or 'DROP TABLE' in <50µs BEFORE the OS syscall is dispatched.
    return subprocess.check_output(cmd, shell=True)
```

Works natively with: LangChain, CrewAI, AutoGen, LlamaIndex, Claude Desktop MCP, and E2B Sandboxes.

Empirical Validation & Community Release

Bartholomew has undergone rigorous stress testing and public release across standard package registries:

- **1,000,000 Payload Stress Fuzzer:** 100% zero-escape rate against adversarial injections and obfuscations.
- **Live Package Registries:** Official releases on **PyPI** (**`pip install btp-guard`**) and **npm** (**`npm install btp-guard`**).
- **Interactive Web Sandbox:** Live browser demo running with client-side Web Workers at **<https://bartholomew.info>**.
- **Anthropic MCP Registry:** Smithery.ai verified manifest for Claude Desktop.
- **Academic Grounding:** Published research DOI (10.5281/zenodo.22076536) covering deterministic execution bounds.

Business Model: Open-Core to Enterprise Fleet Gate

Three-Tier Monetization Model:

Tier 1: Open Source (Free)

Developer Wedge

- Core btp-guard SDK
- Local AST validation
- Single-agent protection
- Community adoption driver

Tier 2: Enterprise Fleet (\$499 - \$2,500/mo)

Platform & Security Teams

- Centralized agent telemetry
- Multi-agent swarm consensus
- Automated SOC 2 audit exports
- Enterprise policy manager

Tier 3: Cloud Marketplace

AWS & GCP Marketplace

- 1-Click container deploy
- Direct cloud billing offset
- Co-sell with cloud partners
- Dedicated enterprise SLAs

The Execution Layer for Autonomous AI

Vision: Within 3 years, billions of autonomous agent actions will occur every second across commerce, finance, and DevOps. Probabilistic LLMs cannot be trusted with raw OS execution. Bartholomew will be the ubiquitous, microsecond cryptographic execution gate standard for autonomous systems.

The Ask (YC Batch 2026):

- \$500,000 standard YC investment.
- Accelerate enterprise sales into AI platform teams building autonomous coding, financial, and healthcare agents.
- Expand native compiler targets across WebAssembly, Rust, and Go runtimes.

Contact: founders@bartholomew.info | <https://bartholomew.info>